

vAPI Scan Comparison

Without Postman Collection vs With Postman Collection

Target: <http://13.59.119.178:8000/vapi>

Date: March 23, 2026

Scanner: Agentic DAST (LLM-driven)

Model: Claude Haiku 4.5 (Bedrock)

1. Executive Summary

This report compares two scans of the vAPI (Vulnerable API) application, which implements the OWASP API Top 10 vulnerabilities across 10 separate API modules (API1-API10). The first scan was conducted without any API documentation, relying entirely on the LLM's autonomous exploration. The second scan was provided with the official Postman collection containing all 24 API endpoints with their request schemas.

Key finding: Providing the Postman collection resulted in 2.6x more findings, 4x more Critical issues, and covered 9 out of 10 OWASP API categories (vs 5/10 without). The scan was also 22% faster and 22% cheaper due to elimination of endpoint discovery overhead.

2. Key Metrics Comparison

Metric	Without Postman	With Postman	Change
Total Findings	66	171	+105 (159%)
Critical	9	36	+27
High	16	63	+47
Medium	17	55	+38
Low	20	17	-3
Info	4	0	-4
Endpoints Crawled	353	197	-156
LLM Calls	595	372	-223 (37%)
Duration (sec)	2562	1999	-563s
Cost (USD)	\$23.28	\$18.07	-\$5.21

3. OWASP API Top 10 Coverage

API	OWASP Vulnerability	Without PM	With PM	Delta
API1	Broken Object Level Authorizat	30 findings	7 findings	-23
API2	Broken Authentication	MISSED	1 findings	NEW
API3	Excessive Data Exposure	9 findings	23 findings	+14
API4	Lack of Resources & Rate Limit	MISSED	6 findings	NEW
API5	Broken Function Level Authoriz	2 findings	18 findings	+16
API6	Mass Assignment	2 findings	4 findings	+2
API7	Security Misconfiguration	2 findings	4 findings	+2
API8	Injection	MISSED	9 findings	NEW
API9	Improper Assets Management	MISSED	1 findings	NEW
API10	Insufficient Logging & Monitor	MISSED	MISSED	same

Coverage Score: 5/10 (without) vs 9/10 (with Postman)

4. Phase-by-Phase Comparison

The following table compares findings discovered in each scan phase.

Phase	Without PM	With PM	Delta
Endpoint discovery	0	0	0
Authentication testing	12	10	-2
Authorization / BOLA	7	14	+7
Endpoint injection testing	0	22	+22
Mass assignment	4	12	+8
Rate limiting	0	8	+8
SSRF testing	0	4	+4
GraphQL testing	1	0	-1
Excessive data exposure	7	13	+6
Business logic	0	10	+10
API Race Conditions	3	0	-3
API Function-Level Authorization	0	0	0
API Host Header Injection	0	0	0
Content-Type Confusion & HTTP Smugglin	12	0	-12
HTTP Method Override	0	0	0
Attack Chain Analysis	6	0	-6

5. Analysis: Why Postman Collection Improves Results

5.1 Eliminated Discovery Overhead

Without the Postman collection, the LLM spent 195 tool calls (33% of total) just on endpoint discovery ? fuzzing paths, guessing request body schemas, and determining authentication mechanisms. With the collection, all 24 endpoints were known upfront, allowing the LLM to allocate 100% of its budget to vulnerability testing.

5.2 Precise Request Schemas

The Postman collection provided exact request bodies (e.g., API4's OTP verify endpoint expects {"otp": "9999"} and API9's v2 login expects {"username": "richardbranson", "pin": "****"}). This enabled targeted attacks: 4-digit PIN brute force for API4, SQL injection on specific fields for API8, and NoSQL operator injection for API3.

5.3 Auth Token Chain Awareness

The collection defined auth token variables ({{api1_auth}}, {{api2_auth}}, etc.) and their relationships. The LLM understood that API1's create user response provides a token used in API1's get/update endpoints, enabling deeper authenticated testing.

5.4 Newly Discovered Attack Vectors

With the collection, the scanner found: SQL injection authentication bypass in API8 (Critical), OTP brute force due to missing rate limiting in API4 (Critical), PIN brute force in API9 v2 (High), and race condition vulnerabilities in API3 and API5 (Critical). None of these were found in the scan without the collection.

6. Conclusion

The Postman collection improved every measurable dimension of the scan: more findings (2.6x), higher severity findings (4x Critical), broader OWASP coverage (9/10 vs 5/10), faster execution (22%), and lower cost (22%). The only API category missed by both scans is API10 (Insufficient Logging & Monitoring), which requires server-side log inspection and cannot be detected externally.

Recommendation: Always provide API documentation (Postman collections, OpenAPI/Swagger specs, or Burp proxy exports) when scanning APIs. The LLM's intelligence is best utilized when it knows the full API surface and can focus on finding vulnerabilities rather than discovering endpoints.